

P201. ¿Por qué es importante tener una comprensión clara de los criterios de finalización de pruebas antes de comenzar la ejecución de las pruebas?

- A. Asegura que todos los miembros del equipo trabajen horas extras hasta que las pruebas estén "terminadas".
- B. Permite que el equipo sepa cuándo se pueden concluir las pruebas de manera objetiva basándose en condiciones predefinidas y medibles.
- C. Garantiza que no se encontrarán defectos después de que finalicen las pruebas.
- D. Reduce la necesidad de documentación de pruebas.

P202. ¿Cómo afecta el concepto de "paradoja del pesticida" al enfoque de mantenimiento de las suites de pruebas automatizadas?

- A. Sugiere que las pruebas automatizadas deberían ejecutarse con menos frecuencia para evitar la fatiga del sistema.
- B. Implica que una vez que se encuentran todos los errores, no se necesitan más pruebas.
- C. Significa que el uso repetido de las mismas pruebas automatizadas ya no encontrará nuevos defectos, por lo que las pruebas deben revisarse y actualizarse regularmente.
- D. Demuestra que las pruebas manuales son siempre superiores a la automatización.

P203. En un proyecto que sigue la metodología ágil, ¿qué adaptación al proceso de prueba fundamental aseguraría pruebas continuas?

- A. Eliminar las fases de planificación y diseño de pruebas por completo.
- B. Realizar todas las pruebas solo al final del lanzamiento.
- C. Integrar el análisis, diseño, implementación y ejecución de pruebas dentro de cada sprint, alineados con las actividades de desarrollo.
- D. Asignar las pruebas exclusivamente a un equipo de control de calidad (QA) separado después de la codificación.

P204. Cuando se rechaza un informe de defecto como "no reproducible", ¿qué pasos de solución de problemas debe tomar el evaluador para proporcionar más evidencia?

- A. Escalar inmediatamente a la alta gerencia.
- B. Volver a probar usando los mismos pasos, entorno, datos, registros y capturas de pantalla exactos, e incluir información contextual detallada.
- C. Asumir que el defecto fue un fallo de una sola vez y cerrarlo.
- D. Culpar al desarrollador por no esforzarse lo suficiente.

P205. ¿Qué buena práctica debe guiar el equilibrio entre la amplitud y la profundidad en la cobertura de pruebas?

- A. Priorizar siempre la profundidad sobre la amplitud.
- B. Maximizar tanto la amplitud como la profundidad por igual, independientemente de las restricciones del proyecto.
- C. Utilizar pruebas basadas en riesgos para enfocar la profundidad en áreas de alto impacto/alta probabilidad, asegurando al mismo tiempo una amplitud mínima en todas las funciones clave.
- D. Permitir que los desarrolladores decidan la cobertura basándose únicamente en la complejidad del código.

P206. ¿Cuál es la diferencia entre una suite de pruebas y un procedimiento de prueba?

- A. Una suite de pruebas es un solo caso de prueba; un procedimiento de prueba es una colección de suites.
- B. Una suite de pruebas es un conjunto de casos de prueba agrupados para su ejecución; un procedimiento de prueba es una secuencia de pasos para ejecutar un caso de prueba.
- C. Una suite de pruebas define los objetivos de prueba; un procedimiento de prueba define los criterios de aprobado/reprobado.
- D. Una suite de pruebas es una colección de casos de prueba organizados para un propósito específico; un procedimiento de prueba es una secuencia documentada de acciones para ejecutar un conjunto de casos de prueba.

P207. ¿Cómo afecta el principio "las pruebas muestran la presencia de defectos" la forma en que se deben reportar los resultados de las pruebas a la gerencia?

- A. Significa que los informes de prueba deben afirmar que el software está libre de defectos si no se encontraron errores.
- B. Justifica omitir las pruebas si el software "se ve bien".
- C. Requiere que los informes de prueba enfatizen que las pruebas exitosas solo reducen el riesgo, no garantizan la corrección, y destaquen las áreas no probadas.
- D. Implica que todos los resultados de las pruebas no son confiables y deben ignorarse.

P208. En un escenario donde una interfaz de usuario cumple con todas las especificaciones, pero viola los estándares de accesibilidad, ¿qué principio de pruebas explica por qué no se identificó este problema antes?

- A. Las pruebas dependen del contexto.
- B. Pruebas tempranas.
- C. La ausencia de errores es una falacia.
- D. Los defectos se agrupan.

P209. Cuando las pruebas de rendimiento muestran tiempos de respuesta aceptables, pero fallan bajo cargas de usuarios concurrentes, ¿qué problema de diseño de pruebas ocurrió probablemente?

- A. Los casos de prueba utilizaron datos de entrada no válidos.
- B. El entorno de pruebas carecía de herramientas de monitoreo.
- C. Los escenarios de prueba de rendimiento se centraron solo en el tiempo de respuesta de un solo usuario y omitieron las pruebas de carga/estrés.
- D. Los evaluadores utilizaron herramientas manuales en lugar de automatizadas.

P210. ¿Qué buena práctica debe seguirse al determinar el nivel apropiado de automatización de pruebas?

- A. Automatizar todos los casos de prueba para eliminar las pruebas manuales.
- B. Evitar la automatización por completo debido al costo de mantenimiento.
- C. Automatizar las pruebas que son repetitivas, estables, de alto valor y propensas a errores humanos, mientras se mantienen manuales las pruebas exploratorias o que cambian con frecuencia.
- D. Dejar que el proveedor de la herramienta decida qué pruebas automatizar.

P211. ¿Por qué es importante tener trazabilidad entre los casos de prueba y las condiciones de prueba?

- A. Permite a los evaluadores reducir el número total de casos de prueba al eliminar duplicados.
- B. Asegura que cada caso de prueba esté vinculado a un requisito o riesgo específico, permitiendo la medición de la cobertura y el análisis de impacto.
- C. Ayuda a los desarrolladores a comprender cómo escribir mejores pruebas unitarias.
- D. Garantiza que todos los defectos se encontrarán durante las pruebas.

P212. ¿Cómo asegura el proceso de prueba fundamental que las pruebas estén integradas con el ciclo de vida general del desarrollo de software?

- A. Ejecutando todos los niveles de prueba simultáneamente, independientemente de la fase del ciclo de vida del desarrollo de software (SDLC).
- B. Alineando las actividades de prueba con las fases y entregables correspondientes del SDLC.
- C. Exigiendo que las pruebas comiencen solo después de que se complete el desarrollo completo del sistema.
- D. Reemplazando las revisiones de desarrollo con la ejecución de pruebas.

P213. En una situación donde los requisitos son incompletos o ambiguos, ¿qué técnicas de análisis de pruebas aún se pueden aplicar de manera efectiva?

- A. Solo esperar hasta que los requisitos estén finalizados antes de comenzar cualquier análisis de pruebas.
- B. Utilizar técnicas basadas en la experiencia y consultar a las partes interesadas para obtener suposiciones.
- C. Confiar únicamente en las suites de regresión automatizadas de versiones anteriores.
- D. Omitir el análisis de pruebas y proceder directamente a la ejecución.

P214. Cuando se corrige un defecto, pero introduce nuevos problemas en funcionalidades que funcionaban anteriormente, ¿qué análisis de causa raíz se debe realizar para mejorar los procesos de desarrollo?

- A. Investigar por qué el evaluador no encontró el defecto original antes.
- B. Analizar la falta de cobertura de pruebas de regresión o una evaluación inadecuada del impacto de los cambios en el flujo de trabajo de desarrollo.
- C. Culpar al desarrollador por sus malas habilidades de codificación.
- D. Asumir que el nuevo problema no está relacionado y tratarlo como un defecto separado.

P215. ¿Qué buena práctica debe guiar la documentación de los procedimientos de configuración del entorno de pruebas?

- A. Documentar solo descripciones de alto nivel para ahorrar tiempo.
- B. Crear instrucciones detalladas, paso a paso y controladas por versión, que incluyan configuraciones de hardware, software, red y datos.
- C. Almacenar las notas de configuración solo en los cuadernos personales de los miembros del equipo.
- D. Asumir que los desarrolladores manejarán la configuración del entorno sin documentación.

P216. ¿Cuál es el propósito principal de las actividades de análisis de pruebas?

- A. Ejecutar casos de prueba y registrar defectos.
- B. Identificar las condiciones de prueba que serán cubiertas por los casos de prueba.

- C. Estimar el costo total del esfuerzo de pruebas.
- D. Seleccionar qué defectos corregir primero.

P217. ¿Cómo influye el principio "los defectos se agrupan" en dónde deben enfocar su atención los evaluadores durante las pruebas exploratorias?

- A. Los evaluadores deben distribuir el tiempo uniformemente entre todas las características para asegurar la equidad.
- B. Los evaluadores deben priorizar los módulos o características con historial de defectos pasado o alta complejidad.
- C. Los evaluadores deben evitar las áreas ya probadas para prevenir la redundancia.
- D. Los evaluadores solo deben explorar nuevas características, ignorando el código heredado.

P218. En un escenario donde una aplicación móvil funciona en la mayoría de las versiones del sistema operativo, pero falla en la última versión, ¿qué principio de pruebas probablemente fue violado?

- A. Las pruebas muestran la presencia de defectos.
- B. Las pruebas exhaustivas son imposibles.
- C. Las pruebas dependen del contexto.
- D. Las pruebas tempranas ahorran tiempo y dinero.

P219. Cuando las partes interesadas solicitan que las pruebas se completen con recursos reducidos, ¿qué factores se deben considerar en la respuesta?

- A. Aceptar inmediatamente para mantener la satisfacción de las partes interesadas.
- B. Realizar una priorización de pruebas basada en riesgos, comunicar las compensaciones y ajustar el alcance o el cronograma de manera transparente.
- C. Reducir las pruebas solo a las suites de regresión automatizadas.
- D. Asignar horas extras a los evaluadores actuales para compensar.

P220. ¿Qué buena práctica se debe implementar para una comunicación efectiva del progreso de las pruebas al equipo del proyecto?

- A. Enviar correos electrónicos diarios que enumeren cada caso de prueba ejecutado.
- B. Proporcionar informes regulares y concisos con métricas como el progreso de las pruebas, las tendencias de defectos, los riesgos y el estado de los criterios de salida.
- C. Compartir registros de defectos sin procesar sin resumir.
- D. Comunicarse solo cuando todas las pruebas estén completas.

P221. ¿Cuál es la diferencia entre calidad y grado en el contexto de las pruebas de software?

- A. La calidad se refiere a cuántas características están incluidas; el grado se refiere a qué tan bien funcionan esas características.
- B. La calidad es el grado en que un sistema satisface las necesidades de las partes interesadas; el grado es una categoría o rango basado en características no funcionales.
- C. Calidad y grado son términos intercambiables en las pruebas de software.
- D. El grado mide el recuento de defectos; la calidad mide la satisfacción del usuario.

P222. ¿Cómo influye la psicología de las pruebas en la forma en que los evaluadores mantienen su objetividad cuando están bajo presión de cronograma?

- A. Los evaluadores deben priorizar la velocidad sobre la precisión para cumplir con los plazos.

- B. Los evaluadores deben evitar reportar defectos menores para reducir la carga de trabajo.
- C. Los evaluadores deben confiar únicamente en las pruebas automatizadas para mantenerse imparciales.
- D. Los evaluadores deben adherirse a criterios predefinidos basados en riesgos y a los objetivos de prueba documentados, independientemente de la presión.

P223. En un proyecto con estrictos requisitos de seguridad, ¿qué adaptación al proceso de prueba fundamental aseguraría pruebas de seguridad integrales?

- A. Omitir el análisis de pruebas e ir directamente a ejecutar pruebas de penetración.
- B. Tratar las pruebas de seguridad como una fase separada después de que se completen las pruebas funcionales.
- C. Integrar el análisis, diseño y configuración del entorno específicos de seguridad en cada etapa del proceso de prueba fundamental.
- D. Confiar solo en auditores externos para la validación de seguridad.

P224. Cuando se encuentra un defecto en producción que no fue cubierto por ningún caso de prueba, ¿qué pasos de solución de problemas se deben tomar para mejorar la cobertura de pruebas futura?

- A. Asignar la culpa al diseñador de pruebas por omitir el escenario.
- B. Agregar inmediatamente un caso de prueba para el defecto específico y continuar.
- C. Realizar un análisis de brechas de requisitos frente a condiciones de prueba, actualizar la matriz de trazabilidad y refinar las técnicas de diseño de pruebas.
- D. Asumir que el defecto fue un caso extremo raro e ignorarlo.

P225. ¿Qué buena práctica debe guiar la selección de técnicas de diseño de pruebas para reglas de negocio complejas?

- A. Usar solo pruebas exploratorias para descubrir fallos de lógica ocultos.
- B. Elegir técnicas basadas en la preferencia o familiaridad del evaluador.
- C. Seleccionar técnicas como tablas de decisión o transición de estados que modelen la lógica y las condiciones explícitamente.
- D. Siempre usar partición de equivalencia para todos los escenarios.

P226. ¿Por qué es importante tener una comprensión clara del alcance de las pruebas antes de comenzar el diseño de las pruebas?

- A. Para asegurar que todos los miembros del equipo tengan los mismos títulos de trabajo.
- B. Para evitar probar características irrelevantes o fuera del alcance, asegurando un uso eficiente de los recursos.
- C. Para garantizar que no se encontrarán defectos fuera de las áreas probadas.
- D. Para permitir que los desarrolladores omitan las pruebas unitarias para los elementos fuera del alcance.

P227. ¿Cómo se aplica el principio de "pruebas tempranas" a las revisiones de diseño?

- A. Las revisiones de diseño son responsabilidad exclusiva de los arquitectos, no de los evaluadores.
- B. Los evaluadores deben esperar hasta que el código esté escrito antes de revisar los diseños.
- C. Las pruebas tempranas significan que los evaluadores ejecutan casos de prueba

durante la fase de diseño.

D. Los evaluadores deben participar en las revisiones de diseño para evaluar la capacidad de prueba y detectar defectos en las especificaciones.

P228. En un escenario donde una aplicación de comercio electrónico procesa los pagos correctamente, pero los cálculos de impuestos son incorrectos para ciertas regiones, ¿qué principio de pruebas explica por qué no se detectó este problema antes?

A. Las pruebas muestran la presencia de defectos.

B. Los defectos se agrupan.

C. Paradoja del pesticida.

D. La ausencia de errores es una falacia.

P229. Cuando los datos de prueba no están disponibles para escenarios críticos, ¿qué enfoques alternativos se pueden utilizar para validar la funcionalidad?

A. Omitir las pruebas de esos escenarios hasta que lleguen los datos reales.

B. Usar datos sintéticos o anonimizados que imiten las condiciones del mundo real, o emplear herramientas de enmascaramiento/generación de datos.

C. Asumir que la funcionalidad funciona si las pruebas unitarias pasan.

D. Confiar en los datos de producción copiados directamente en los entornos de prueba.

P230. ¿Qué buena práctica se debe seguir al realizar un análisis de causa raíz para tipos de defectos recurrentes?

A. Enfocarse solo en el defecto más reciente para ahorrar tiempo.

B. Atribuir las causas a errores de miembros individuales del equipo.

C. Utilizar métodos estructurados como los 5 Porqués o diagramas de espina de pescado para identificar debilidades sistémicas de proceso o diseño.

D. Aceptar que algunos defectos siempre reaparecerán y no hacer nada.

P231. ¿Cuál es la diferencia entre una carta de prueba y un script de prueba?

A. Una carta de prueba define instrucciones detalladas paso a paso; un script de prueba es una declaración de misión de alto nivel para pruebas exploratorias.

B. Una carta de prueba es un objetivo de alto nivel para una sesión de prueba; un script de prueba contiene pasos específicos predefinidos y resultados esperados.

C. Ambos términos son intercambiables para el código de prueba automatizado.

D. Un script de prueba se usa solo en pruebas manuales; una carta solo para automatización.

P232. ¿Cómo afecta el concepto de "ausencia de errores es una falacia" la forma en que las pruebas complementan las pruebas de aceptación del usuario?

A. Significa que, si no se encuentran errores, el software está listo para su lanzamiento.

B. Enfatiza que no encontrar defectos no garantiza que el software cumpla con las necesidades del usuario o sea utilizable.

C. Implica que las pruebas de aceptación del usuario son innecesarias si las pruebas del sistema pasaron.

D. Sugiere que solo los desarrolladores deben realizar las pruebas de aceptación del usuario para ahorrar tiempo.

P233. En un proyecto con múltiples integraciones a sistemas de terceros, ¿qué enfoque de diseño de pruebas sería más efectivo?

- A. Partición de equivalencia solo para campos de entrada.
- B. Pruebas de tablas de decisión para reglas de negocio.
- C. Pruebas de transición de estados para flujos de trabajo de la interfaz de usuario.
- D. Pruebas de interfaz e integración utilizando análisis de valores límite, adivinación de errores y validación de contratos.

P234. Cuando se encuentra un defecto crítico durante la fase final de pruebas, ¿qué factores se deben considerar para decidir si retrasar el lanzamiento?

- A. Solo la disponibilidad del equipo de desarrollo para corregir el error.
- B. El impacto del defecto en los usuarios, la disponibilidad de soluciones alternativas, el riesgo comercial, la estabilidad de la corrección y los plazos de lanzamiento.
- C. Si el evaluador que encontró el error es senior.
- D. Retrasar automáticamente el lanzamiento independientemente del contexto.

P235. ¿Qué buena práctica debe guiar la gestión de los entornos de prueba en una infraestructura basada en la nube?

- A. Configurar manualmente cada entorno bajo demanda sin documentación.
- B. Usar infraestructura como código para aprovisionar y gestionar entornos de manera reproducible y controlada por versiones.
- C. Compartir un entorno entre todos los equipos para reducir costos.
- D. Evitar el uso de datos similares a producción para ahorrar almacenamiento.

P236. ¿Cuál es el propósito principal de la especificación de diseño de pruebas?

- A. Documentar los resultados finales de las pruebas y el estado de aprobado/reprobado.
- B. Definir condiciones de prueba, casos de prueba y datos de prueba basados en la base de pruebas.
- C. Asignar evaluadores a módulos específicos.
- D. Reemplazar la necesidad de ejecución de pruebas.

P237. ¿Cómo afecta el principio "las pruebas dependen del contexto" al nivel de detalle en los casos de prueba?

- A. Todos los casos de prueba deben ser altamente detallados, independientemente del tipo de proyecto.
- B. El detalle del caso de prueba debe adaptarse al riesgo del proyecto, la experiencia del equipo, las necesidades regulatorias y el modelo de ciclo de vida.
- C. Solo los casos de prueba automatizados necesitan detalle; los manuales pueden ser vagos.
- D. El nivel de detalle debe ser decidido únicamente por el gerente del proyecto.

P238. En un escenario donde una aplicación de salud funciona correctamente para condiciones comunes, pero falla para casos médicos raros, ¿qué actividades de prueba fueron probablemente insuficientes?

- A. Pruebas de rendimiento bajo carga.
- B. Pruebas de seguridad para el cifrado de datos.
- C. Análisis de valores límite en los campos de entrada.
- D. Pruebas de casos extremos y clases de equivalencia específicas del dominio para condiciones médicas raras.

P239. Cuando las partes interesadas cuestionan por qué se necesitan pruebas para un lanzamiento de corrección de errores, ¿cómo debe justificar un gerente de

pruebas el esfuerzo de prueba?

- A. Porque la política de la empresa exige pruebas para todos los lanzamientos.
- B. Porque las correcciones de errores pueden introducir regresiones o afectar características no relacionadas; las pruebas de regresión específicas aseguran la estabilidad.
- C. Porque los evaluadores necesitan algo que hacer.
- D. Porque el desarrollador lo solicitó.

P240. ¿Qué buena práctica se debe implementar para el mantenimiento efectivo de casos de prueba en un sistema en evolución?

- A. Nunca actualizar los casos de prueba: crear otros nuevos para cada cambio.
- B. Archivar todos los casos de prueba antiguos y comenzar de nuevo con cada lanzamiento.
- C. Revisar, refactorizar y retirar los casos de prueba regularmente en función de los requisitos, el riesgo y el uso cambiantes.
- D. Asignar el mantenimiento de los casos de prueba solo a evaluadores junior.

P241. ¿Por qué es importante distinguir entre las actividades de monitoreo y control de pruebas?

- A. El monitoreo recopila datos; el control toma acciones correctivas basadas en esos datos.
- B. Ambas actividades son idénticas y se pueden realizar indistintamente.
- C. El monitoreo lo realizan los desarrolladores; el control lo realizan los evaluadores.
- D. El control implica escribir casos de prueba; el monitoreo implica ejecutarlos.

P242. ¿Cómo influye la psicología de las pruebas en la forma en que los evaluadores abordan el análisis de valores límite?

- A. Los evaluadores evitan las pruebas de límites porque son demasiado complejas.
- B. Los evaluadores se centran solo en entradas válidas para complacer a los desarrolladores.
- C. Los evaluadores adoptan una mentalidad escéptica, investigando deliberadamente los casos extremos donde es probable que haya errores.
- D. Los evaluadores confían en los desarrolladores para identificar los límites.

P243. En un proyecto con integración de sistemas heredados, ¿qué adaptación al proceso de prueba fundamental aseguraría pruebas efectivas?

- A. Omitir el análisis de pruebas e ir directamente a la ejecución para ahorrar tiempo.
- B. Usar solo pruebas de regresión automatizadas de proyectos anteriores.
- C. Mejorar el análisis de pruebas para incluir especificaciones de interfaz, mapeo de datos y escenarios de manejo de errores entre sistemas.
- D. Confiar únicamente en las pruebas de aceptación del usuario para la validación.

P244. Cuando se corrige un defecto, pero el mismo problema aparece en un flujo de trabajo diferente, ¿qué técnica de análisis de causa raíz se debe aplicar?

- A. Análisis de Pareto.
- B. Diagrama de Ishikawa (Espina de Pescado).
- C. Priorización de casos de prueba.
- D. Agrupación de defectos.

P245. ¿Qué buena práctica debe guiar la documentación de los resultados de las pruebas para el cumplimiento normativo?

- A. Almacenar solo el estado de aprobado/reprobado para reducir el papeleo.
- B. Documentar los resultados de las pruebas de manera informal en los registros de chat del equipo.
- C. Mantener registros completos y a prueba de manipulaciones que incluyan casos de prueba, registros de ejecución, marcas de tiempo y personal involucrado.
- D. Compartir los resultados verbalmente en las revisiones del sprint.

P246. ¿Cuál es la diferencia entre los criterios de finalización de pruebas y los criterios de salida?

- A. Los criterios de finalización definen cuándo detener las pruebas; los criterios de salida definen cuándo comenzar.
- B. Son sinónimos y se usan indistintamente en todos los estándares.
- C. Los criterios de finalización son objetivos a nivel de proyecto; los criterios de salida son condiciones medibles específicas de la fase.
- D. Los criterios de finalización son informales; los criterios de salida son siempre automatizados.

P247. ¿Cómo afecta el principio "las pruebas exhaustivas son imposibles" al enfoque de pruebas de algoritmos complejos?

- A. Los equipos deben omitir las pruebas de algoritmos por completo.
- B. Las pruebas deben confiar únicamente en las pruebas unitarias de los desarrolladores.
- C. Las pruebas deben utilizar una selección de entradas basada en riesgos, partición de equivalencia y análisis de límites para maximizar la detección de defectos con casos mínimos.
- D. Todas las combinaciones de entrada posibles deben probarse utilizando automatización a escala en la nube.

P248. En un escenario donde una aplicación financiera cumple con todos los requisitos funcionales, pero carece de un manejo adecuado de errores para entradas no válidas, ¿qué principio de pruebas explica por qué no se identificó este problema?

- A. Las pruebas muestran la presencia de defectos.
- B. La ausencia de errores es una falacia.
- C. Pruebas tempranas.
- D. Los defectos se agrupan.

P249. Cuando la ejecución de pruebas se ve afectada por dependencias externas que están fuera del control del equipo de pruebas, ¿qué enfoque de solución de problemas se debe utilizar para minimizar el impacto?

- A. Cancelar todos los casos de prueba dependientes hasta que se solucione la dependencia.
- B. Culpar públicamente al proveedor externo.
- C. Usar virtualización de servicios o mocks para simular dependencias externas y continuar con las pruebas.
- D. Esperar indefinidamente a que la dependencia esté disponible.

P250. ¿Qué buena práctica se debe seguir al establecer un programa de mejora del proceso de pruebas basado en lecciones aprendidas?

- A. Implementar todas las sugerencias inmediatamente sin validación.
- B. Centrarse solo en las actualizaciones de herramientas, ignorando a las personas y los procesos.
- C. Utilizar un modelo estructurado para priorizar las mejoras basándose en brechas medibles y objetivos comerciales.
- D. Asignar las tareas de mejora solo al personal junior.

P251. ¿Cuál es la diferencia principal entre los modelos de ciclo de vida de desarrollo de software secuenciales e iterativos desde una perspectiva de pruebas?

- A. Los modelos secuenciales prueban solo al final; los modelos iterativos prueban continuamente durante todo el desarrollo.
- B. Los modelos secuenciales utilizan pruebas manuales; los modelos iterativos requieren automatización.
- C. Los modelos iterativos eliminan la necesidad de pruebas de sistema.
- D. Los modelos secuenciales permiten un diseño temprano de pruebas; los modelos iterativos retrasan la planificación de pruebas.

P252. ¿Cómo difiere el enfoque de pruebas entre las metodologías de desarrollo en cascada y ágil?

- A. La cascada utiliza pruebas exploratorias; el Ágil se basa en casos de prueba scripteados.
- B. Las pruebas en cascada se basan en fases y ocurren después del desarrollo; las pruebas Ágiles son continuas e integradas con el desarrollo.
- C. El Ágil elimina las pruebas de regresión; la cascada las requiere en gran medida.
- D. Ambos utilizan procesos de documentación y aprobación idénticos.

P253. En un escenario donde una aplicación bancaria se desarrolla utilizando un modelo secuencial, ¿qué desafíos específicos podrían enfrentar los evaluadores durante la fase de pruebas de sistema?

- A. Los evaluadores tienen demasiadas oportunidades para influir en las decisiones de diseño.
- B. Los requisitos pueden estar desactualizados o mal interpretados, lo que lleva a un retrabajo importante cuando se encuentran defectos tarde.
- C. Los entornos de prueba siempre están listos antes de que comiencen las pruebas de sistema.
- D. Los desarrolladores están fácilmente disponibles para corregir problemas inmediatamente durante las pruebas.

P254. Cuando se encuentra un defecto crítico durante las pruebas de aceptación del usuario en un ciclo de vida de desarrollo de software (SDLC) secuencial, ¿qué pasos de solución de problemas se deben tomar para minimizar el impacto en el proyecto?

- A. Lanzar el software inmediatamente y corregir el defecto en la siguiente versión.
- B. Documentar el defecto, evaluar el impacto, involucrar a las partes interesadas en una junta de control de cambios y evaluar las opciones de reversión o parche en caliente.
- C. Asignar la culpa al equipo de pruebas por no haberlo encontrado antes.
- D. Ignorar el defecto si afecta solo a un pequeño grupo de usuarios.

P255. ¿Qué buena práctica se debe implementar para la planificación de pruebas en un modelo de desarrollo iterativo?

- A. Crear un plan de pruebas integral al inicio del proyecto y nunca actualizarlo.
- B. Desarrollar una estrategia de pruebas de alto nivel al principio y luego refinar los planes de pruebas detallados por iteración basándose en los elementos del backlog.
- C. Aplazar toda la planificación de pruebas hasta la iteración final.
- D. Asignar la planificación de pruebas exclusivamente a los evaluadores sin la opinión de los desarrolladores.

P256. ¿Por qué son particularmente importantes las pruebas de regresión en los enfoques de desarrollo incremental?

- A. Porque cada incremento agrega nueva funcionalidad que podría romper las características existentes.
- B. Porque los modelos incrementales no realizan pruebas unitarias.
- C. Para satisfacer a los auditores que requieren informes de pruebas mensuales.
- D. Porque los evaluadores tienen tiempo libre entre incrementos.

P257. ¿Cómo le explicaría el concepto de "desplazamiento a la izquierda" en las pruebas a un gerente de proyecto que no está familiarizado con las prácticas ágiles?

- A. Significa mover las actividades de prueba hacia el lado derecho del cronograma para una mayor precisión.
- B. Implica comenzar las pruebas más temprano en el ciclo de vida del desarrollo para encontrar y corregir defectos más pronto, reduciendo el costo y el riesgo.
- C. Se refiere a usar evaluadores zurdos para una mayor creatividad.
- D. Significa que los evaluadores deben reportar al lado izquierdo del organigrama.

P258. En una situación donde los equipos de desarrollo y pruebas trabajan en silos en un modelo secuencial, ¿qué estrategia de comunicación se alinea con una integración efectiva del ciclo de vida de desarrollo de software (SDLC)?

- A. Mantener una separación estricta para preservar la integridad de los roles.
- B. Realizar revisiones multifuncionales periódicas de requisitos, diseño y casos de prueba antes de que comience la codificación.
- C. Esperar hasta las pruebas de sistema para compartir cualquier comentario con los desarrolladores.
- D. Usar solo el correo electrónico para todas las comunicaciones para mantener pistas de auditoría.

P259. Cuando la ejecución de pruebas se retrasa constantemente en un modelo de desarrollo iterativo, ¿qué enfoque de solución de problemas se debe tomar para mejorar la alineación con el desarrollo?

- A. Extender la duración de la iteración indefinidamente para acomodar las pruebas.
- B. Analizar los cuellos de botella, aumentar la automatización de pruebas e involucrar a los evaluadores más temprano en la planificación del sprint.
- C. Culpar a los evaluadores por la mala gestión del tiempo.
- D. Omitir las pruebas de regresión para ponerse al día.

P260. ¿Qué buena práctica debe guiar el momento de la configuración del entorno de pruebas en relación con los hitos de desarrollo?

- A. Configurar el entorno de pruebas solo después de que todo el desarrollo esté completo.
- B. Provisionar y validar el entorno de pruebas en paralelo con el desarrollo, idealmente antes de que comience la codificación.
- C. Usar el entorno de producción para todas las pruebas para ahorrar costos.
- D. Permitir que los desarrolladores configuren el entorno de pruebas a medida que codifican.

P261. ¿Cuál es la diferencia entre las pruebas de componente y las pruebas de integración en términos de alcance y objetivos?

- A. Las pruebas de componente validan flujos de trabajo de negocio de extremo a extremo; las pruebas de integración verifican funciones individuales.
- B. Las pruebas de componente se centran en módulos o clases individuales de forma aislada; las pruebas de integración verifican las interacciones entre componentes integrados.
- C. Las pruebas de componente requieren el despliegue completo del sistema; las pruebas de integración se ejecutan en las estaciones de trabajo de los desarrolladores.
- D. Ambos tipos de prueba tienen como objetivo encontrar defectos en la interfaz de usuario.

P262. ¿Cómo influye el modelo de "cuadrantes de pruebas" en la planificación de pruebas en entornos ágiles?

- A. Exige que todas las pruebas se automaticen en los Cuadrantes 1 y 2.
- B. Clasifica las pruebas por orientación tecnológica frente a orientación al negocio y apoyo frente a crítica del producto, guiando la selección del tipo de prueba y las responsabilidades del equipo.
- C. Reemplaza la necesidad de niveles de prueba como las pruebas de sistema o de aceptación.
- D. Define fases de prueba fijas que deben completarse en orden.

P263. En un proyecto que utiliza la metodología Scrum con sprints de dos semanas, ¿qué adaptación al proceso de prueba fundamental sería más efectiva para asegurar la calidad?

- A. Retrasar todas las pruebas hasta el final del lanzamiento.
- B. Realizar todos los niveles de prueba dentro de cada sprint, con retroalimentación continua y automatización.
- C. Asignar las pruebas solo a los dos últimos días de cada sprint.
- D. Eliminar la planificación de pruebas para aumentar la velocidad.

P264. Cuando un defecto encontrado durante las pruebas de sistema requiere un retrabajo significativo en un proyecto ágil, ¿qué enfoque de solución de problemas se debe utilizar para evaluar el impacto en el sprint?

- A. Eliminar inmediatamente la historia de usuario del backlog del sprint.
- B. Ignorar el defecto hasta el siguiente sprint para evitar la interrupción.
- C. Realizar un análisis de impacto colaborativo con el Dueño del Producto para decidir si corregirlo ahora, ajustar el alcance o diferirlo.
- D. Exigir a los desarrolladores que trabajen horas extras para corregirlo sin discusión.

P265. ¿Qué buena práctica se debe seguir al determinar el nivel apropiado de automatización de pruebas para diferentes modelos de ciclo de vida de desarrollo

de software (SDLC)?

- A. Automatizar todos los casos de prueba independientemente del modelo para maximizar la cobertura.
- B. Usar la misma estrategia de automatización para proyectos en cascada y ágiles.
- C. Adaptar el alcance de la automatización al SDLC: en ágil, priorizar la retroalimentación rápida; en secuencial, centrarse en la regresión y la automatización a nivel de sistema.
- D. Evitar la automatización en modelos secuenciales debido a las pruebas tardías.

P266. ¿Por qué es crítico el concepto de "definición de terminado" para las actividades de pruebas en metodologías ágiles?

- A. Permite a los evaluadores omitir la documentación para ahorrar tiempo.
- B. Asegura que cada historia de usuario cumpla con los estándares de calidad acordados, incluidas las tareas de prueba, antes de considerarse completa.
- C. Reemplaza la necesidad de planificación y diseño de pruebas.
- D. Garantiza cero defectos en cada incremento.

P267. ¿Cómo impacta la comprensión del ciclo de vida del desarrollo de software en la selección de los niveles de prueba apropiados?

- A. No tiene impacto: los niveles de prueba son fijos independientemente del SDLC.
- B. Permite a los evaluadores omitir las pruebas de sistema en proyectos ágiles.
- C. Permite la alineación de los niveles de prueba con las fases del proyecto, los entregables y el momento del riesgo.
- D. Exige que las pruebas de aceptación siempre ocurran al final.

P268. En un escenario donde una aplicación de salud pasa del desarrollo a las pruebas de sistema, pero los requisitos críticos siguen siendo ambiguos, ¿qué problema fundamental del ciclo de vida de desarrollo de software (SDLC) está presente?

- A. Falta de preparación del entorno de pruebas.
- B. Pruebas unitarias insuficientes por parte del desarrollador.
- C. Ingeniería y validación de requisitos inadecuadas antes de que comience el desarrollo.
- D. Mala comunicación entre evaluadores y operaciones.

P269. Cuando las partes interesadas cuestionan por qué las pruebas continúan después de que el desarrollo está "completo" en un modelo secuencial, ¿cómo debe justificar un gerente de pruebas las actividades de prueba en curso?

- A. Explicar que los desarrolladores a menudo ocultan errores intencionalmente.
- B. Declarar que las pruebas son una obligación contractual y deben realizarse.
- C. Aclarar que "desarrollo completo" solo significa que la codificación ha terminado; las pruebas validan la funcionalidad, el rendimiento, la seguridad y la usabilidad contra los requisitos.
- D. Prometer que las pruebas se omitirán en proyectos futuros para ahorrar tiempo.

P270. ¿Qué buena práctica se debe implementar para transferencias efectivas entre los equipos de desarrollo y pruebas en un ciclo de vida de desarrollo de software (SDLC) secuencial?

- A. Exigir que los desarrolladores escriban todos los casos de prueba antes de la transferencia.
- B. Usar informes verbales informales para transferir el conocimiento.

- C. Establecer criterios de salida/entrada formales y documentados para cada transición de fase.
- D. Permitir que las pruebas comiencen solo después de la integración completa del sistema.

P271. ¿Cuál es el propósito principal de las pruebas de aceptación en relación con las partes interesadas del negocio?

- A. Validar que el sistema cumple con las especificaciones técnicas definidas por los arquitectos.
- B. Asegurar que el software esté libre de todos los defectos antes del lanzamiento.
- C. Confirmar que el sistema satisface los requisitos del negocio y está listo para su uso operativo.
- D. Evaluar el rendimiento bajo condiciones de carga máxima.

P272. ¿Cómo impacta el principio de integración continua en el momento y el alcance de las pruebas de integración?

- A. Las pruebas de integración se aplazan hasta que todos los módulos estén completos para reducir la complejidad.
- B. Las pruebas de integración se realizan solo una vez por ciclo de lanzamiento.
- C. Las pruebas de integración se ejecutan de manera continua y automática con cada confirmación de código.
- D. Las pruebas de integración son reemplazadas por completo por las pruebas unitarias en los pipelines de CI.

P273. En una situación donde el entorno de pruebas difiere significativamente de producción en un proyecto ágil, ¿qué problemas potenciales podrían surgir y cómo deberían abordarse?

- A. No surgen problemas: los equipos ágiles confían en las pruebas unitarias para compensar.
- B. Los defectos pueden omitirse o reportarse falsamente debido a discrepancias ambientales; abordarlos alineando los entornos de prueba y producción lo más posible.
- C. Los equipos ágiles deben omitir la validación del entorno para ahorrar tiempo.
- D. Usar datos de producción exclusivamente en las pruebas para garantizar la precisión.

P274. Cuando un proceso de negocio crítico falla después de un cambio menor de código en un entorno DevOps, ¿qué técnica de análisis de causa raíz sería más apropiada?

- A. Análisis FODA.
- B. Los 5 Porqués.
- C. Análisis PESTLE.
- D. Simulación Monte Carlo.

P275. ¿Qué buena práctica debe guiar el equilibrio entre los esfuerzos de pruebas automatizadas y manuales en los diferentes modelos de ciclo de vida de desarrollo de software (SDLC)?

- A. Automatizar todas las pruebas para eliminar el error humano.
- B. Usar pruebas manuales solo para sistemas heredados y automatización para nuevas características.
- C. Elegir la automatización para pruebas repetitivas, estables y de alto valor; usar pruebas

manuales para escenarios exploratorios, de usabilidad y ad-hoc.

D. Evitar la automatización en Ágil; confiar únicamente en las pruebas manuales.

P276. ¿Por qué es importante que los evaluadores participen en las sesiones de refinamiento en metodologías ágiles?

A. Para asignar tareas de prueba a los desarrolladores durante la planificación del sprint.

B. Para asegurar que las historias de usuario sean comprobables, claras e incluyan criterios de aceptación.

C. Para demostrar los casos de prueba completados al dueño del producto.

D. Para asumir la propiedad de la priorización del backlog.

P277. ¿Cómo cambia el enfoque DevOps el concepto tradicional de pruebas de sistema?

A. Las pruebas de sistema se eliminan porque las pruebas unitarias cubren todo.

B. Las pruebas de sistema se convierten en una compuerta única antes de la producción.

C. Las pruebas de sistema se integran en el pipeline y se ejecutan continuamente con bucles de retroalimentación rápidos.

D. Las pruebas de sistema son realizadas solo por los equipos de operaciones después del despliegue.

P278. En un escenario donde una aplicación móvil funciona perfectamente en desarrollo, pero falla en producción debido a diferencias de infraestructura, ¿qué problema de integración del ciclo de vida de desarrollo de software (SDLC) ocurrió probablemente?

A. Falta de cobertura de pruebas unitarias.

B. Ausencia de análisis de código estático.

C. Paridad de entorno inadecuada entre las etapas de desarrollo, prueba y producción.

D. Uso excesivo de pruebas manuales.

P279. Cuando un informe de defecto carece de información suficiente para su reproducción en un entorno ágil, ¿qué pasos debe tomar el evaluador para mejorar el informe?

A. Cerrar el defecto y asumir que fue un fallo de una sola vez.

B. Volver a probar, capturar pasos detallados, registros, capturas de pantalla y contexto del entorno, luego actualizar el informe.

C. Asignar el defecto al desarrollador y pedirle que lo resuelva.

D. Esperar a que el defecto vuelva a ocurrir en el próximo sprint.

P280. ¿Qué buena práctica se debe seguir al priorizar los casos de prueba para una revisión de sprint con tiempo limitado?

A. Ejecutar todos los casos de prueba en orden aleatorio para asegurar la equidad.

B. Priorizar los casos de prueba basándose en el riesgo, el valor comercial y el impacto del cambio.

C. Probar solo las nuevas características e ignorar la regresión.

D. Ejecutar solo las pruebas automatizadas para ahorrar tiempo.

P281. ¿Por qué las pruebas de componente son típicamente realizadas por desarrolladores en lugar de evaluadores dedicados?

A. Los desarrolladores tienen mejor acceso a los servidores de producción.

B. Los evaluadores carecen de habilidades de programación para comprender la lógica

del código.

- C. Los desarrolladores entienden la estructura interna y la lógica del componente, lo que permite pruebas de caja blanca efectivas.
- D. Los evaluadores son demasiado costosos para involucrarlos en las etapas tempranas.

P282. ¿Cómo asegura el proceso de prueba fundamental una cobertura completa en los diferentes niveles de prueba?

- A. Ejecutando los mismos casos de prueba en cada nivel de prueba.
- B. Definiendo objetivos de prueba, técnicas y criterios de entrada/salida distintos para cada nivel de prueba.
- C. Asignando todas las pruebas a un solo equipo para mantener la consistencia.
- D. Omitiendo los niveles de prueba inferiores si los superiores pasan.

P283. En un proyecto que utiliza la metodología Kanban, ¿qué adaptación a la planificación de pruebas sería más efectiva para gestionar el flujo continuo?

- A. Crear un plan maestro de pruebas rígido y basado en fases que cubra todo el trabajo futuro.
- B. Definir ciclos de prueba estáticos alineados con los lanzamientos mensuales.
- C. Utilizar una estrategia de pruebas viva y liviana que evolucione con el flujo de trabajo y los límites de trabajo en progreso.
- D. Eliminar la planificación de pruebas por completo para maximizar la velocidad.

P284. Cuando un evaluador identifica un defecto que los desarrolladores consideran de baja prioridad a pesar del impacto comercial en un equipo ágil, ¿qué estrategia de comunicación se debe emplear?

- A. Escalar inmediatamente al dueño del producto con evidencia del impacto comercial.
- B. Insistir en que el defecto se corrija inmediatamente, independientemente del consenso del equipo.
- C. Aceptar el criterio de los desarrolladores para mantener la armonía del equipo.
- D. Registrar el defecto y nunca volver a mencionarlo.

P285. ¿Qué buena práctica debe guiar la creación de datos de prueba que representen escenarios de uso del mundo real en las diferentes fases del ciclo de vida de desarrollo de software (SDLC)?

- A. Usar solo datos sintéticos para evitar problemas de privacidad.
- B. Reutilizar el mismo pequeño conjunto de datos en todas las fases para mantener la consistencia.
- C. Derivar los datos de prueba de datos de producción anonimizados y enriquecerlos para cubrir condiciones límite y de error.
- D. Permitir que los desarrolladores generen datos de prueba ad-hoc durante la codificación.

P286. ¿Cuál es la diferencia entre las pruebas funcionales y no funcionales en términos de cuándo deben realizarse en el ciclo de vida de desarrollo de software (SDLC)?

- A. Las pruebas funcionales se realizan solo en las pruebas de sistema; las no funcionales solo en la aceptación.
- B. Las pruebas no funcionales deben esperar hasta que todo el sistema esté integrado; las funcionales pueden comenzar antes.

- C. Las pruebas funcionales pueden comenzar a nivel de componente; las pruebas no funcionales son principalmente a nivel de sistema, pero deben considerarse desde el principio.
- D. Todas las pruebas deben aplazarse hasta las pruebas de aceptación del usuario (UAT) para ahorrar tiempo.

P287. ¿Cómo influye el principio de "fallar rápido" en DevOps en el diseño de las suites de pruebas automatizadas?

- A. Alienta a ejecutar todas las pruebas solo después del despliegue completo en producción.
- B. Prioriza las pruebas rápidas y livianas al principio del pipeline para detectar problemas rápidamente.
- C. Elimina las pruebas manuales por completo para acelerar la entrega.
- D. Requiere que todas las pruebas pasen antes de que se escriba cualquier código.

P288. En un escenario donde las pruebas automatizadas pasan en el pipeline de CI/CD pero los usuarios reportan problemas en producción, ¿qué principio de integración del ciclo de vida de desarrollo de software (SDLC) explica esta discrepancia?

- A. Falta de independencia de las pruebas.
- B. Cobertura de pruebas inadecuada.
- C. Brecha de paridad del entorno entre los entornos de prueba y producción.
- D. Mala comunicación entre evaluadores y desarrolladores.

P289. Cuando un entorno de pruebas carece de componentes de hardware críticos presentes en producción en un proyecto ágil, ¿qué riesgos potenciales introduce esto y cómo deberían mitigarse?

- A. No hay riesgos: el comportamiento del software es independiente del hardware.
- B. Mayor riesgo de defectos de integración o rendimiento no detectados; mitigar mediante el uso de simuladores de hardware o réplicas en la nube.
- C. El riesgo es mínimo; los desarrolladores pueden probar las interacciones de hardware manualmente más tarde.
- D. Los evaluadores deben omitir los casos de prueba relacionados con el hardware por completo.

P290. ¿Qué buena práctica se debe implementar para procesos efectivos de revisión de casos de prueba en un equipo ágil distribuido?

- A. Realizar revisiones solo al final del sprint.
- B. Utilizar revisiones asíncronas por pares con listas de verificación claras y herramientas colaborativas.
- C. Asignar un revisor por caso de prueba para evitar confusiones.
- D. Omitir las revisiones para mantener la velocidad.

P291. ¿Por qué es importante distinguir entre las actividades de pruebas de sistema y pruebas de aceptación?

- A. Las pruebas de sistema validan las especificaciones técnicas; las pruebas de aceptación validan la preparación para el negocio y la satisfacción del usuario.
- B. Ambas prueban la misma funcionalidad pero con diferentes herramientas.
- C. Las pruebas de aceptación son realizadas solo por desarrolladores; las pruebas de

sistema por evaluadores.

D. Las pruebas de sistema ocurren después de las pruebas de aceptación en el ciclo de vida.

P292. ¿Cómo incorpora el proceso de prueba fundamental los enfoques de pruebas basadas en riesgos en los diferentes modelos de ciclo de vida de desarrollo de software (SDLC)?

A. Las pruebas basadas en riesgos solo son aplicables en proyectos en cascada.

B. Las fases de análisis y diseño de pruebas priorizan las condiciones de prueba en función de la probabilidad y el impacto de la falla, independientemente del SDLC.

C. Las pruebas basadas en riesgos reemplazan todas las demás técnicas de prueba en los modelos iterativos.

D. Solo los riesgos de seguridad se consideran en las pruebas basadas en riesgos.

P293. En una situación donde los evaluadores y los desarrolladores tienen diferentes interpretaciones de los requisitos en una reunión de planificación del sprint, ¿qué pasos se deben tomar para resolver la ambigüedad?

A. Los evaluadores deben aceptar la interpretación del desarrollador para evitar conflictos.

B. Escalar inmediatamente al CEO para una decisión final.

C. Aclarar con el Dueño del Producto, actualizar los criterios de aceptación y documentar el entendimiento acordado.

D. Omitir la escritura de pruebas para esa historia hasta el próximo sprint.

P294. Cuando se encuentra un defecto crítico al final del sprint, ¿qué enfoque de solución de problemas se debe utilizar para evaluar el impacto en el objetivo del sprint?

A. Extender inmediatamente el plazo del sprint para corregir el defecto.

B. Ignorar el defecto si no bloquea el despliegue.

C. Evaluar la gravedad del defecto, evaluar la viabilidad de la corrección en el tiempo restante y consultar al Dueño del Producto para la repriorización o el aplazamiento.

D. Asignar la culpa al evaluador por no haberlo encontrado antes.

P295. ¿Qué buena práctica debe guiar la documentación de los resultados de las pruebas para las reuniones de revisión del sprint?

A. Proporcionar solo un resumen de aprobado/reprobado para mantenerlo simple.

B. Compartir registros de defectos detallados con seguimientos de pila y fragmentos de código.

C. Presentar métricas claras y concisas vinculadas a los objetivos del sprint y las historias de usuario.

D. Omitir los resultados de las pruebas, ya que el incremento del producto habla por sí mismo.

P296. ¿Cuál es el objetivo principal de las actividades de pruebas de mantenimiento?

A. Rediseñar la interfaz de usuario para una mejor usabilidad.

B. Verificar que los cambios no introduzcan nuevos defectos y que el sistema siga funcionando como se espera.

C. Reemplazar todos los casos de prueba existentes por otros nuevos.

D. Reducir el número de evaluadores en el proyecto.

P297. ¿Cómo influye el concepto de pruebas de "desplazamiento a la izquierda" en las actividades de recopilación de requisitos?

- A. Retrasa la recopilación de requisitos hasta después de la planificación de pruebas.
- B. Alienta a los evaluadores a participar en las revisiones de requisitos para mejorar la claridad, la capacidad de prueba y la integridad.
- C. Elimina la necesidad de documentación formal de requisitos.
- D. Requiere que los desarrolladores escriban todos los requisitos sin la opinión del evaluador.

P298. En un escenario donde una aplicación financiera calcula el interés correctamente para la mayoría de las cuentas, pero falla para cuentas de alto saldo durante las pruebas de sistema, ¿qué fase del ciclo de vida de desarrollo de software (SDLC) probablemente omitió este escenario?

- A. Fase de codificación.
- B. Fase de pruebas unitarias.
- C. Fase de requisitos y diseño de pruebas.
- D. Fase de despliegue.

P299. Cuando las partes interesadas cuestionan el valor de las pruebas exploratorias en un sprint ágil, ¿cómo debe justificar un gerente de pruebas su inclusión en la estrategia de pruebas?

- A. Las pruebas exploratorias reemplazan todas las pruebas scripteadas y son más rápidas.
- B. Permite el descubrimiento rápido de defectos de alto impacto en áreas complejas o mal especificadas donde las pruebas scripteadas pueden pasar por alto problemas.
- C. Reduce la necesidad de documentación de pruebas por completo.
- D. Solo los evaluadores junior deben realizar pruebas exploratorias.

P300. ¿Qué buena práctica se debe seguir al hacer la transición de las pruebas de integración a las pruebas de sistema?

- A. Comenzar las pruebas de sistema inmediatamente sin verificar los criterios de salida de las pruebas de integración.
- B. Asegurar que todos los criterios de salida de las pruebas de integración se cumplan y que los defectos de interfaz principales se resuelvan antes de comenzar las pruebas de sistema.
- C. Omitir las pruebas de integración si las pruebas de sistema cubrirán todo.
- D. Entregar todos los entornos de pruebas a los desarrolladores durante la transición.